

Architectural Calibration of Roo Code Indexing and Model Context Protocol Infrastructure for Educational Knowledge Graphs

The convergence of autonomous artificial intelligence agents, advanced data orchestration, and semantic vector indexing presents unprecedented opportunities for analyzing unstructured, highly complex domain data. Within the specific architectural boundaries of the `kings_college_galway` repository, the deployment of the `sruth/oideachais` (education data stream) and `sruth/bonneagar` (infrastructure stream) projects demands a perfectly synchronized development environment.¹ The overarching objective of this infrastructure is to ingest, parse, and semantically map decades of pedagogical documentation from the Irish State Examinations Commission (SEC) and the National Council for Curriculum and Assessment (NCCA).¹ To achieve this within the stringent constraints of a 48-hour Z.ai Pro Coding Plan sprint, developers rely heavily on the Roo Code Visual Studio Code extension.¹

Roo Code serves as the primary interface for autonomous agentic workflows, utilizing semantic codebase indexing to grant foundation models a deep, contextual understanding of the repository structure.² However, integrating this local indexing mechanism with remote Google Gemini embedding APIs and the highly specialized Z.ai Model Context Protocol (MCP) servers frequently results in critical system instability.⁴ The emergence of perpetual indexing loops, catastrophic API rate limiting, and dimensional mismatch errors actively threatens the viability of the educational knowledge graph project.⁵ Maximizing the utility of the MCP servers and stabilizing the Roo Code environment requires an exhaustive, forensic calibration of the extension's underlying configuration parameters, vector storage paradigms, and agentic operational constraints.

Deconstructing the Semantic Codebase Indexing Paradigm

The fundamental premise of Roo Code's codebase indexing is to transform linear text into a highly dimensional, mathematically queryable vector space. Rather than relying on simple lexical grep searches or keyword matching, the indexing system allows the agent to execute natural language queries—such as searching for specific data extraction logic or database connection handling—and retrieve semantically related code blocks.³ This is an absolute necessity for navigating the `sruth/oideachais` pipeline, which contains a dense matrix of asynchronous Python scrapers, Dagster Software-Defined Assets (SDAs), and complex relational schemas mapping NCCA learning outcomes to SEC grading rubrics.¹

When the indexing feature is enabled, Roo Code does not blindly ingest the entire repository as a single continuous string. Instead, it relies on an intelligent parsing framework utilizing Tree-sitter.³ Tree-sitter generates an Abstract Syntax Tree (AST) of the supported programming languages within the workspace, allowing the indexer to identify distinct semantic blocks such as classes, individual methods, and discrete function definitions.³ For documentation files, such as the Architectural Decision Records housed within the docs/data_engineering directory, the system utilizes Markdown headers as semantic entry points.¹

A pervasive misconception surrounding this architecture is the concept of "index length." In classical relational database administration, index length refers to the byte size of the key utilized in an indexed sequential access method.⁸ However, in the context of Roo Code's AI-native vector embeddings, "index length" is a misnomer that conflates chunk size constraints with vector dimensionality.³ The chunking granularity within Roo Code is strictly hardcoded within the extension's internal processing logic.⁹ The system mandates a minimum block size of 100 characters and a maximum block size of 1,000 characters.³ When the Tree-sitter parser encounters a function that exceeds this 1,000-character upper bound, it does not arbitrarily slice the string at the character limit; rather, it intelligently fragments the code at the nearest logical boundary.³ Because these chunk sizes are hardcoded, developers cannot manually adjust an "index length" parameter to force the ingestion of larger, contiguous code blocks.⁹ Furthermore, the parser implements a strict file size limitation, automatically bypassing any individual file that exceeds one megabyte.³

This hardcoded limitation is critically important for the kings_college_galway repository. The bunchloch directory acts as the staging ground for raw, unparsed PDF binaries retrieved from the examination portals, many of which far exceed the one-megabyte threshold and lack AST-compatible semantic structures.¹ Attempting to force the Roo Code indexer to process these raw educational corpora will invariably lead to severe performance degradation and excessive API consumption.¹⁰

The Dimensionality Bottleneck and Gemini Embedding Instability

The semantic transformation of these code chunks requires an external embedding provider. While Roo Code supports a myriad of providers, the integration with Google Gemini presents unique dimensional challenges that often manifest as silent indexing failures. Google provides multiple generations of embedding models via its API, each with distinct mathematical properties and output dimensionalities.⁶

The older generation model, text-embedding-004, outputs a vector with 768 dimensions.⁶ This relatively compact vector footprint is highly stable and fully compatible with the standard initialization sequences of the local Qdrant vector database utilized by Roo Code.⁶ Conversely, Google's state-of-the-art embedding model, gemini-embedding-001, utilizes a sophisticated technique known as Matryoshka Representation Learning (MRL).¹¹ MRL allows the model to theoretically scale its output dimensions, but it defaults to a massive 3072 dimensions.⁶ While

this profound dimensionality is exceptional for capturing the intricate semantic nuances of bilingual Irish educational texts or complex physics syllabi, it introduces severe instability within the extension.¹

Empirical analysis indicates that configuring Roo Code to utilize gemini-embedding-001 with its native 3072 dimensions consistently results in immediate indexing crashes, often surfacing as generic network errors or "fetch failed" notifications in the extension logs.⁶ The root cause is a vector dimension mismatch within the Qdrant collections.³ If a developer initiates the index with a 768-dimension model and subsequently switches to the more powerful 3072-dimension model without completely purging the existing database, the Qdrant instance will reject the incoming mathematical payloads.³

To ensure absolute stability when embedding the sruth/oideachais and sruth/bonneagar pipelines, the infrastructure must be standardized. The embedding provider must be explicitly set to Google Gemini, utilizing the text-embedding-004 model to guarantee the stable 768-dimension output.⁶ Any deviation from this standard necessitates a complete erasure of the local index data via the extension's configuration popover to prevent catastrophic collection mismatches.³

Embedding Model	Default Dimensionality	Matryoshka Representation Learning	Stability within Roo Code / Qdrant	Recommendation for sruth/oideachais
text-embedding-004	768 Dimensions	Unsupported	Highly Stable	Recommended. Ensures consistent vector ingestion without triggering collection dimension mismatch errors. ⁶
gemini-embedding-001	3072 Dimensions	Supported	Unstable / High Failure Rate	Avoid. Causes immediate "fetch failed" network errors and collection rejections unless meticulously configured. ⁶

Diagnosing the Perpetual Indexing Loop and Rate Limit Exhaustion

The most debilitating technical hurdle encountered in this environment is the emergence of a perpetual indexing loop that rapidly exhausts the Gemini API rate limits. This phenomenon effectively paralyzes the autonomous agent, preventing it from executing the semantic

extraction tasks required for the educational knowledge graph.⁵ This failure cascade is a direct consequence of asynchronous concurrency overwhelming the provider's infrastructure, combined with inadequate error handling and state management within the Roo Code orchestration layer.⁵

When Roo Code initiates a full workspace scan, the internal `DirectoryScanner` class aggregates the fragmented code blocks and dispatches them to the embedding API in concurrent batches.¹³ By default, the extension hardcodes this process to an `embeddingBatchSize` of 60, operating with a high concurrency threshold.¹³ When these massive, highly concurrent payload matrices strike the Gemini API endpoints, they almost instantaneously breach the strict Requests Per Minute (RPM) and Tokens Per Minute (TPM) limits associated with the free or baseline tiers of Google AI Studio.¹⁵

Upon breaching these quotas, the Gemini backend responds with an HTTP 429 Resource Exhausted error, or occasionally an HTTP 413 Payload Too Large error, explicitly stating that the validation batch size of 60 is greater than the maximum allowed.¹⁵ In a resilient distributed system, the client application would parse the `Retry-After` headers provided in the 429 response payload and execute an exponential backoff algorithm, pausing execution until the API bucket refreshes.¹⁸

However, the current iteration of the Roo Code indexer exhibits a critical vulnerability in its handling of 429 errors.⁵ When the Gemini provider rejects the batch, the extension frequently fails to persist the indexing progress.⁵ The system state resets, and the user interface either freezes, displays a continuous yellow pulsing indicator, or silently aborts the operation entirely.¹³ Because the progress cache is destroyed, the next automated retry attempts to re-index the repository from the very first block.⁵ Because the API key remains locked under the 429 rate limit, this subsequent attempt fails instantly, creating an aggressive, infinite loop that burns through any remaining daily quota without successfully embedding a single file.⁵

The Qdrant Persistence Vulnerability

This looping behavior is drastically exacerbated by how the local Qdrant vector database is deployed. Roo Code supports connecting to a local, free Docker instance of Qdrant.³ If the developer instantiates this Docker container without explicitly configuring persistent volume mounts mapping the container's storage back to the host filesystem, the Qdrant database operates entirely in ephemeral memory.²¹

The codebase indexing orchestration logic relies on cryptographic hashing (SHA-256) to perform incremental scans. When the IDE is launched, the orchestrator queries the vector database; if it finds existing data, it compares the current filesystem hashes against the stored hashes, processing only the net-new or modified files.²¹ However, if the Qdrant container was spun down or the machine rebooted without volume persistence, the entire semantic index evaporates.²¹ Roo Code detects an empty database and automatically initiates a full, brute-force re-index of the entire `kings_college_galway` repository upon startup.²¹ For a project containing thousands of Python execution scripts, Dagster definitions, and extraction schemas, this immediate, unprompted mass-indexing event practically guarantees a collision with the Gemini

API rate limits, trapping the developer in the 429 loop before actual development work can even commence.²¹

Architectural Calibration via settings.json

To permanently resolve the API throttling loops and stabilize the IDE environment, the data engineering team must implement precise, surgical overrides within the VS Code settings.json file. These parameters synthetically throttle the token velocity generated by the extension, ensuring that the internal batching logic remains subordinate to the provider's infrastructural limits.

The most critical intervention is the modification of the `roo-cline.codeIndex.embeddingBatchSize` parameter.¹⁴ By default, this value is set to 60.¹⁴ This massive batch size is the primary catalyst for the HTTP 413 and HTTP 429 validation failures.¹⁷ The infrastructure architect must manually reduce this integer to 10.¹⁷ While decreasing the batch size exponentially increases the total processing time required to complete a full repository scan, it drastically reduces the instantaneous memory load and token throughput, ensuring a slow, stable, and ultimately successful ingestion of the sruth/oideachais pipelines.¹⁴

Secondly, the scope of the index must be artificially constrained using the `roo-cline.maximumIndexedFilesForFileSearch` directive.¹⁴ The default configuration permits the indexing of up to 10,000 files.¹⁴ In complex monorepos or projects heavily reliant on localized virtual environments and nested dependencies, the indexer will exhaust API quotas attempting to embed irrelevant library code.¹⁴ Reducing this ceiling to 5,000 files ensures that the system focuses its computational resources exclusively on the bespoke educational extraction logic rather than standard boilerplate.¹⁴

Finally, to guarantee that these critical overrides are consistently applied across all developer environments and are not inadvertently overwritten during troubleshooting, the project should leverage the `roo-cline.autoImportSettingsPath` configuration.¹⁴ By defining an absolute path to a master JSON configuration file, the extension will automatically load the throttled batch sizes and file limits upon initialization.¹⁴ This declarative approach to settings management ensures absolute consistency across the federated development team.

VS Code settings.json Key	Default Value	Calibrated Value	Architectural Justification
<code>roo-cline.codeIndex.embeddingBatchSize</code>	60	10	Mitigates HTTP 413 and HTTP 429 API rejections by restricting the volume of code chunks transmitted in a single network request. ¹⁴
<code>roo-cline.maximumIndexedFilesForFileSearch</code>	10000	5000	Imposes a hard ceiling on file ingestion,

			preventing API exhaustion on deep, irrelevant dependency trees within the sruth/bonneagar infrastructure. ¹⁴
roo-cline.autoImportSettingsPath	""	"/path/to/master.json"	Guarantees immutable configuration loading on startup, ensuring agentic environments remain synchronized with the required API throttling measures. ¹⁴

Shielding the Context Window with Strict .rooignore Protocols

While throttling the API request velocity prevents acute network failures, it does not solve the fundamental issue of mathematical waste. Applying semantic embeddings to raw, static data is an anti-pattern that leads to severe context poisoning.²⁴ The foundation model's context window—even one as massive as the GLM-5.1's 200,000-token capacity—must be fiercely protected from irrelevant noise to maintain its reasoning efficacy.¹

The kings_college_galway repository architecture is unique in that it houses both the executable ETL logic (within sruth/oideachais) and the resulting data artifacts.¹ The asynchronous web scrapers continuously pull massive PDF binaries representing Leaving Certificate marking schemes and Junior Cycle syllabi into the bunchloch directory.¹ Furthermore, the CocolIndex incremental processing framework outputs highly structured JSON representations of these documents.¹

If the Roo Code indexer is permitted to parse these directories, it will squander the Gemini API quota attempting to generate vector embeddings for tens of thousands of static examination questions and binary file metadata.¹⁰ To prevent this, the data engineer must implement a rigorous .rooignore file at the root of the workspace.²⁶

The .rooignore file operates using identical glob pattern syntax to a standard .gitignore file, dictating exactly which directories and file extensions the autonomous agent is forbidden from accessing.²⁶ The engine strictly enforces these rules across multiple capabilities; if a directory is ignored, the agent cannot parse it for the codebase index, nor can it utilize the read_file or apply_diff tools to manipulate its contents.²⁶

For the educational knowledge graph project, the .rooignore file must explicitly exclude the following structural components:

- **Raw Binary Storage:** /bunchloch/**/* .pdf must be ignored to prevent the indexer from choking on massive, non-semantic document artifacts.¹

- **Extraction Outputs:** `/data/**/*`.json or any downstream storage vectors must be ignored to prevent the recursive re-embedding of data that has already been processed by the production CocolIndex flow.¹
- **Orchestration State:** `/.dagster/` and `/.dlt/` must be excluded to shield the agent from the highly volatile state management databases and event logs generated by the data orchestration pipelines.¹

By isolating the Roo Code indexer purely to the active Python executables and the theoretical Architectural Decision Records within `docs/data_engineering`, the system curates a pristine, highly relevant semantic index.¹ This ensures that when the developer queries the agent regarding "Dagster asset materialization," the retrieved context is strictly the implementation code, free from the polluting noise of thousands of raw examination questions.

Delineating Roo Code Indexing from CocolIndex Processing

It is an absolute necessity for the infrastructure team to understand the fundamental distinction between the local Roo Code indexing feature and the CocolIndex incremental processing framework utilized within the `sruth/oideachais` pipeline. Both systems employ semantic chunking and vector embeddings, but their architectural purposes are entirely divergent.

Roo Code's indexing is a localized, IDE-bound mechanism designed exclusively to augment the AI agent's awareness of the codebase architecture.³ It exists solely to help the agent write better Python code and understand the repository's directory structure.

Conversely, CocolIndex is the production-grade data transformation engine driving the actual educational knowledge graph.¹ Within the Dagster orchestration flow, CocolIndex is responsible for detecting net-new SEC examination PDFs, executing Tree-sitter chunking specifically tailored to pedagogical layouts, and generating the high-dimensional vector embeddings that will eventually be queried by educators and autonomous grading systems.¹ CocolIndex relies on its own internal state tracking and lineage mechanisms, ensuring that transformations are incredibly fast and incrementally updated without re-processing the entire corpus.²⁷

The local IDE must never attempt to perform the duties of the production pipeline. The Roo Code indexer must be explicitly firewalled via `.rooignore` from the data assets managed by CocolIndex, ensuring that the two vectorization engines operate in complete, non-interfering parallel.

Synergizing the Architecture with the Z.ai MCP Ecosystem

With the local IDE environment constrained, stabilized, and shielded from runaway API loops, the developer can fully exploit the massive potential of the Z.ai Pro Coding Plan. The defining characteristic of this subscription tier is its integration with the Model Context Protocol (MCP), an open-standard architecture that allows Large Language Models to interact securely with remote environments, specialized APIs, and complex local filesystems via standardized JSON-RPC 2.0 messages.¹

The deployment of these MCP servers fundamentally supercharges the autonomous agent's

capabilities, allowing it to transcend the limitations of local text parsing and static training data. However, the execution of these protocols is heavily regulated by the strict quotas inherent to the Z.ai Pro Plan.

The infrastructure implements a shared monthly quota pool of 1,000 calls encompassing the Web Search, Web Reader, and Zread MCP servers.¹ Conversely, the Vision MCP Server draws directly from the core LLM's dynamic 5-hour rolling pool of approximately 400 prompts.¹ Furthermore, these prompt deductions are subject to aggressive temporal multipliers; usage during peak hours (14:00 to 18:00 UTC+8) consumes quota at three times (3x) the standard rate.¹ Optimizing the 48-hour development sprint requires a highly tactical, mathematically calculated deployment of these MCP capabilities.

Zread MCP: Remote Repository Comprehension

The Zread MCP Server represents a profound paradigm shift in agentic development. Operating over a Streamable-HTTP/SSE transport protocol, Zread provides the GLM-5.1 foundation model with deep, remote repository comprehension tools, specifically `get_repo_structure`, `read_file`, and `search_doc`.¹

For publicly hosted repositories like `kings_college_galway`, the Zread MCP essentially bypasses the necessity for local codebase indexing.³¹ Rather than relying on the volatile Gemini embeddings and the local Qdrant database to understand the project architecture, the agent can invoke Zread to query the repository structure dynamically via the remote server.³⁰

When the developer tasks the agent with constructing a new asynchronous web scraper for the National Council for Curriculum and Assessment, the agent first utilizes the Zread `search_doc` tool to analyze the existing data governance policies housed within `docs/data_engineering`.¹ It then executes `get_repo_structure` to understand the hierarchy of the `sruth/oideachais` Dagster pipelines.¹ This guarantees absolute architectural fidelity; the newly generated code will perfectly mirror the project's established stylistic paradigms, utilizing the correct environmental variables, logging standards, and asset definitions. Because Zread API calls draw from the monthly 1,000-call MCP pool rather than the 5-hour dynamic prompt pool, this strategy fiercely protects the core inferential capacity of the LLM for heavy reasoning tasks, completely sidestepping the local 429 API throttling loops.¹

Web Reader and Web Search: Dynamic Ontological Construction

The pedagogical philosophy underpinning the Irish educational system is currently undergoing a massive structural evolution. The redevelopment of the Senior Cycle (2025-2029) is actively shifting the curriculum away from rigid content lists toward highly nuanced, hierarchical Learning Outcomes defined by pedagogical action verbs (e.g., "Critique," "Assess," "Investigate").¹ To construct an accurate knowledge graph that mathematically maps an SEC examination question to a theoretical NCCA learning outcome, the underlying relational schema must reflect these current policies.

The Web Search MCP Server (`webSearchPrime`) allows the autonomous agent to pierce its static knowledge cutoff.¹ During the architectural planning phase, the agent utilizes this remote

tool to actively query the internet for the most recent NCCA consultation reports and finalized curriculum specifications regarding the Senior Cycle redevelopment.¹ This ensures the resulting ontological matrices generated within docs/data_engineering are factually flawless. Simultaneously, the Web Reader MCP Server (webReader) provides the agent with the capacity to fetch full-page content and deeply nested structured metadata from specific URLs.¹ The archival structure of the State Examinations Commission portal (examinations.ie) utilizes complex, heavily nested routing protocols based on examination year, subject type, and academic level.¹ The agent employs the Web Reader to ingest the raw Document Object Model (DOM) of these portals, analyzing the HTML hierarchy and identifying the hidden endpoints necessary to bypass the terms-and-conditions acceptance gates.¹ By analyzing the DOM, the agent autonomously codes highly resilient asynchronous Python scrapers using standard libraries such as httpx or aiohttp.¹ Crucially, once the agent has generated and validated these executable scraping scripts, the architecture mandates an immediate cessation of Web Reader MCP usage. By transitioning the mass-downloading phase entirely to the local Python execution environment, the project entirely circumvents the 1,000-call monthly quota limit imposed on the MCP servers.¹ This tactical withdrawal preserves the remaining MCP resources for emergency debugging or complex documentation retrieval later in the lifecycle.

Vision MCP: High-Fidelity Pedagogical Extraction

The ultimate objective of the oideachais pipeline is the structured semantic extraction of examination materials.¹ This represents the most formidable computational challenge of the entire endeavor. Examination papers, particularly those concerning Higher Level Mathematics, Physics, or Geography, are incredibly visually dense.¹ They rely on complex algebraic matrices, calculus notations, topographical maps, and intricate technical flowcharts.¹

Furthermore, many of these examinations are published bilingually, presenting the Irish (Gaeilge) text directly adjacent to the English translation in complex, multi-column layouts.¹ Standard Optical Character Recognition (OCR) engines and naive PDF-to-Markdown parsers invariably fail when confronted with this complexity. They scramble the columns, separate mathematical equations from their identifying question numbers, and utterly destroy the semantic integrity of the evaluation rubrics.

To overcome this, the architecture dynamically routes complex documents to the local Vision MCP Server. Operating locally via standard input/output (stdio) and drawing directly from the 400-prompt rolling quota, the Vision MCP utilizes the cutting-edge GLM-4.6V visual language model.¹ The model employs tools such as extract_text_from_screenshot and understand_technical_diagram to perform flawless, high-fidelity structural extraction.¹

When the CocolIndex incremental processor encounters a document with a high visual density score, it slices the PDF into visual frames and passes them to the Vision MCP.¹ The GLM-4.6V model translates the intricate grids, the highly subjective examiner notation, and the specialized A-scale penalty rubrics into perfectly formatted, machine-readable Markdown tables.¹ This preserves the exact spatial relationships and mathematical logic required to evaluate student

performance, an absolute prerequisite for feeding the data into the GLM-5.1 model for final semantic synthesis.

MCP Server	Protocol / Transport	Z.ai Quota Source	Strategic Role within the Educational Pipeline
Zread MCP	HTTP/SSE	Shared 1,000 calls/month	Replaces local indexing by providing remote repository comprehension, ensuring new code matches existing sruth/oideachais architecture. ¹
Web Reader MCP	HTTP/SSE	Shared 1,000 calls/month	Analyzes the DOM structure of examinations.ie to inform the generation of asynchronous Python scrapers. ¹
Web Search MCP	HTTP/SSE	Shared 1,000 calls/month	Retrieves real-time policy updates regarding the NCCA Senior Cycle redevelopment to ensure accurate schema mapping. ¹
Vision MCP	Stdio (Local)	400 Prompts / 5-Hours	Executes high-fidelity OCR on complex, bilingual SEC marking schemes, converting mathematical rubrics into structured Markdown. ¹

Agentic Orchestration and Operational Harmonization

The technological synergy achieved by stabilizing the Roo Code indexer and integrating the Z.ai MCP ecosystem must be explicitly codified into the operational behavior of the autonomous agents. Roo Code supports the configuration of bespoke personas via a workspace-level .roomodes file, which dictates the strict operational mandates, tool access permissions, and required knowledge bases for specific agentic roles.¹

The .roomodes file within the kings_college_galway repository currently defines four distinct

specialists: the Data Engineer, the AI Engineer, the Frontend Developer, and the DevOps Architect.¹ All four personas share an identical baseline of tool group permissions, granting them universal access to read, edit, browser, command, and the crucial mcp tools.¹ However, their interaction with the environment is governed by mandatory indexing paths targeting domain-specific architectural patterns.

The AI Engineer and MCP-Builder Constraints

The 🧠 **AI Engineer** (ai-engineer) persona operates as the central intelligence orchestrator of the data pipeline.¹ Its primary mandate is the bridging of unstructured educational data into highly structured schemas using agentic workflows, OCR, and semantic extraction.¹

The .roomodes file explicitly tasks this persona with "MCP Server development".¹ When the AI Engineer is commanded to design the dynamic routing protocols between the CocolIndex execution flow and the local Vision MCP Server, the configuration enforces a mandatory context retrieval requirement. The agent is strictly commanded to consult the localized domain knowledge housed within the .skills/mcp-builder, .skills/baml, and .skills/litellm directories.¹

This structural requirement acts as an architectural safeguard. By forcing the agent to index these specific instructional subdirectories prior to generating code, the system guarantees that the AI Engineer utilizes strict type-enforcement (via BoundaryML) when transmitting extraction prompts to the GLM-5.1 foundation model. This enforces absolute relational integrity, ensuring that the resulting JSON payloads perfectly mirror the expected database schemas and preventing catastrophic failures during downstream data insertion.

Isolating Qdrant from the LanceDB Production Environment

A profound architectural distinction must be maintained when interacting with the 📊 **Data Engineer** (data-engineer) persona. This role is tasked with managing the project's ELT pipelines and is responsible for maintaining the "federated DuckLake architecture" using tools like Dagster and DuckDB.¹ Crucially, the mode is explicitly commanded to focus on "managing vector embeddings" and must consult the guidelines within the .skills/lancedb directory.¹

This configuration reveals a bifurcated approach to vector storage within the repository. The Roo Code extension relies natively on the Qdrant vector database to power its local IDE codebase indexing, enabling the agent to understand the Python scripts and markdown documentation.³

Conversely, the actual production pipeline within sruth/oideachais utilizes LanceDB as its primary analytical storage mechanism for housing the high-dimensional semantic representations of the NCCA syllabi and SEC marking schemes.¹

The infrastructure architect must ensure these two vector ecosystems remain completely conceptually and physically isolated. Qdrant is the epistemological engine for the IDE; LanceDB is the production repository for the knowledge graph. The .rooignore file must aggressively exclude the actual LanceDB .lance storage directories.²⁶ Failure to isolate these environments will result in the Roo Code extension attempting to recursively embed the massive production vector database—an action that would immediately overwhelm system memory, trigger fatal timeouts, and instantly exhaust the Gemini API quotas.²⁵

Orchestrating Around Z.ai Temporal Multipliers

The final component of operational harmonization involves aligning the execution schedules of the Dagster pipelines with the temporal multipliers enforced by the Z.ai Pro Coding Plan. The GLM-5.1 model is an immensely powerful, highly resource-intensive foundation model. To manage global server load, Z.ai dictates that usage during peak hours (14:00 to 18:00 UTC+8) consumes the dynamic prompt quota at an accelerated rate of three times (3x) the standard metric.¹

The  **DevOps Architect** (devops-architect) persona—responsible for CI/CD deployments and managing the tunneling between local processing units and remote infrastructure like Hetzner and PlanetScale—must encode these temporal realities into the automated execution graphs.¹ The most computationally expensive phase of the project—the semantic extraction process where CocolIndex concatenates massive markdown representations of an examination paper, its associated marking scheme, and the relevant syllabus into a single payload for the GLM-5.1 model—must be rigidly cron-scheduled.¹ The Dagster orchestrator must be configured to automatically pause all LLM-intensive API calls during the 14:00 to 18:00 UTC+8 peak window.¹ During this four-hour blackout period, the pipeline relies exclusively on localized execution. The infrastructure executes the asynchronous Python scrapers to hoard raw PDF binaries, performs baseline Markdown conversions using high-speed local parsers like Docling, and queues the intermediate artifacts in the local file system.¹

Once the off-peak window resumes and the deduction multiplier drops to 2x (or to 1x during promotional periods), the Dagster orchestrator aggressively flushes the queue.¹ Utilizing the GLM-5.1 model's massive 200,000-token context window, the system processes hundreds of pages of educational material in massive, highly optimized inference passes, returning perfectly synthesized relational JSON arrays.¹ By aligning the agentic execution strategy with the economic realities of the API provider, the infrastructure team can effectively triple the total volume of data parsed, synthesized, and inserted into the LanceDB knowledge graph before the 48-hour subscription window expires.

Final Recommendations for the Kings College Galway Architecture

The challenges encountered while deploying the sruth/oideachais and sruth/bonneagar pipelines—namely the perpetual indexing loops and the Gemini API throttling—are highly symptomatic of a semantic infrastructure that has outgrown its default configuration parameters. By executing surgical overrides within the VS Code settings.json file, the token velocity directed toward the Gemini endpoints can be successfully throttled. Reducing the embeddingBatchSize to 10 and artificially lowering the maximumIndexedFilesForFileSearch ceiling forces the local extension to respect the strict validation bounds of the provider, completely eliminating the catastrophic HTTP 413 and HTTP 429 failure cascades.

Furthermore, applying rigorous glob patterns within a workspace .rooignore file shields the localized Qdrant indexer from the massive static binaries and downstream JSON extraction

artifacts generated by the CocolIndex processors. This prevents context poisoning, dramatically accelerates IDE responsiveness, and mathematically guarantees that API quotas are utilized solely for analyzing active, executable code.

Ultimately, the architectural apex is achieved by transitioning the burden of repository comprehension away from local parsing tools and onto the remote Z.ai Model Context Protocol ecosystem. Leveraging the Streamable-HTTP Zread MCP server grants the GLM-5.1 model instantaneous, quota-efficient understanding of the project's complex topological structures. Simultaneously, deploying the local Stdio Vision MCP allows the agent to conquer the visual density of the Irish educational examinations, translating complex mathematical grids and bilingual rubrics into pristine, machine-readable formats.

By harmonizing these advanced protocols with the specialized personas defined in the .roomodes configuration, the kings_college_galway infrastructure evolves from a series of disjointed data scripts into a highly cohesive, self-regulating, and economically optimized intelligence platform capable of fundamentally deciphering the pedagogical architectures of modern education.

Works cited

1. AI for Exam Paper Analysis.pdf
2. Roo Code: A Guide With 7 Practical Examples - DataCamp, accessed April 25, 2026, <https://www.datacamp.com/tutorial/roo-code>
3. Codebase Indexing | Roo Code Documentation, accessed April 25, 2026, <https://docs.roocode.com/features/codebase-indexing>
4. Roo code codebase indexing is so slow : r/RooCode - Reddit, accessed April 25, 2026, https://www.reddit.com/r/RooCode/comments/1mrsdpj/roo_code_codebase_indexing_is_so_slow/
5. Codebase Indexing Bug - Indexing progress completely erased after 429 code from the Gemini provider · Issue #6650 · RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026, <https://github.com/RooCodeInc/Roo-Code/issues/6650>
6. Codebase Indexing fails when using gemini-embedding-001 (3072 dimensions), works with text-embedding-004 (768 dimensions) · Issue #5774 · RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026, <https://github.com/RooCodeInc/Roo-Code/issues/5774>
7. Build a Real-Time Codebase Index in 5 Minutes with CocolIndex (Rust + Tree-sitter), accessed April 25, 2026, <https://dev.to/badmonster0/build-a-real-time-codebase-index-in-5-minutes-with-cocolindex-rust-tree-sitter-eo3>
8. .\| US! RI>A COMPUTER PROGRAM SUMMARIES - UNT Digital Library, accessed April 25, 2026, https://digital.library.unt.edu/ark:/67531/metadc868150/m2/1/high_res_d/4076732.pdf
9. The codebase indexing is performing incorrect code parsing. · Issue #5247 - GitHub, accessed April 25, 2026,

- <https://github.com/RooCodeInc/Roo-Code/issues/5247>
10. There should be a toggle that doesn't index my repository by default ! :
r/RooCode - Reddit, accessed April 25, 2026,
https://www.reddit.com/r/RooCode/comments/1lxxn27/there_should_be_a_toggle_that_doesnt_index_my/
 11. Gemini Embedding now generally available in the Gemini API - Google Developers Blog, accessed April 25, 2026,
<https://developers.googleblog.com/gemini-embedding-available-gemini-api/>
 12. Issue Title: Bug: Codebase indexing fails with "fetch failed" when using Gemini embedder · Issue #6626 · RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026, <https://github.com/RooCodeInc/Roo-Code/issues/6626>
 13. Codebase indexing stops at ~300 blocks out of 65k with OpenAI text-embedding-3-large, cannot restart · Issue #5819 · RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026,
<https://github.com/RooCodeInc/Roo-Code/issues/5819>
 14. Import, Export, and Reset Settings | Roo Code Documentation, accessed April 25, 2026, <https://docs.roocode.com/features/settings-management>
 15. Troubleshooting guide | Gemini API - Google AI for Developers, accessed April 25, 2026, <https://ai.google.dev/gemini-api/docs/troubleshooting>
 16. Issue with 429 Error on Gemini API Despite Staying Within Rate Limits, accessed April 25, 2026,
<https://discuss.ai.google.dev/t/issue-with-429-error-on-gemini-api-despite-staying-within-rate-limits/66584>
 17. How to edit Roo Code's VScode settings · Issue #138 · wecode-ai/RunVSAgent - GitHub, accessed April 25, 2026,
<https://github.com/wecode-ai/RunVSAgent/issues/138>
 18. Gemini-embedding-001 not respecting free quota limits · Issue #5713 · RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026,
<https://github.com/RooCodeInc/Roo-Code/issues/5713>
 19. Learn how to handle 429 resource exhaustion errors in your LLMs | Google Cloud Blog, accessed April 25, 2026,
<https://cloud.google.com/blog/products/ai-machine-learning/learn-how-to-handle-429-resource-exhaustion-errors-in-your-llms>
 20. [BUG] Codebase indexing and embedding is extremely slow and unstable on low-end hardware (local and cloud models, including OpenAI-compatible local endpoints) · Issue #8875 · RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026, <https://github.com/RooCodeInc/Roo-Code/issues/8875>
 21. [BUG] Roo code reindex my large codebase everytime I open my repo #12145 - GitHub, accessed April 25, 2026,
<https://github.com/RooCodeInc/Roo-Code/issues/12145>
 22. Full reindexing after reboot : r/RooCode - Reddit, accessed April 25, 2026,
https://www.reddit.com/r/RooCode/comments/1n0k95o/full_reindexing_after_reboot/
 23. [BUG] Repository Indexing - Status - Error - Initial Scan Failed: Indexing failed: Batch processing failed after 3 attempts: Bad Reques · Issue #10721 ·

- RooCodeInc/Roo-Code - GitHub, accessed April 25, 2026, <https://github.com/RooCodeInc/Roo-Code/issues/10721>
24. Working with Large Projects | Roo Code Documentation, accessed April 25, 2026, <https://docs.roocode.com/advanced-usage/large-projects>
 25. Roo Code keeps burning API calls just to re-read files : r/RooCode - Reddit, accessed April 25, 2026, https://www.reddit.com/r/RooCode/comments/1nj7pyv/roo_code_keeps_burning_api_calls_just_to_reread/
 26. Using .rooignore to Control File Access | Roo Code Documentation, accessed April 25, 2026, <https://docs.roocode.com/features/rooignore>
 27. Incremental Processing with CocolIndex, accessed April 25, 2026, <https://cocolindex.io/blogs/incremental-processing>
 28. Building Intelligent Codebase Indexing with CocolIndex: A Deep Dive into Semantic Code Search - Medium, accessed April 25, 2026, <https://medium.com/@cocolindex.io/building-intelligent-codebase-indexing-with-cocolindex-a-deep-dive-into-semantic-code-search-e93ae28519c5>
 29. A Developer's Guide to the MCP - Zep, accessed April 25, 2026, <https://www.getzep.com/ai-agents/developer-guide-to-mcp/>
 30. Zread MCP Server - Overview - Z.AI DEVELOPER DOCUMENT, accessed April 25, 2026, <https://docs.z.ai/devpack/mcp/zread-mcp-server>
 31. Search vs Reader vs Zread: A Claude Code Guide to Z.ai MCP Servers - AI @ Sulat.com, accessed April 25, 2026, <https://ai.sulat.com/search-vs-reader-vs-zread-a-claude-code-guide-to-z-ai-mcp-servers-134cece1ad96>
 32. Web Reader MCP Server - Overview - Z.AI DEVELOPER DOCUMENT, accessed April 25, 2026, <https://docs.z.ai/devpack/mcp/reader-mcp-server>
 33. Vision MCP Server - Overview - Z.AI DEVELOPER DOCUMENT, accessed April 25, 2026, <https://docs.z.ai/devpack/mcp/vision-mcp-server>